Group 1:
Matthew Barback
Hudson Byers
Ryan Megna
Victor Paulino
Matthew Petrarca

Database Views and Normalization

1. Document the exact SQL queries you used to define each view. Include a brief explanation of each view's purpose and expected results.

View 1: vw_function_lookup

Purpose: Serves as the primary full-detail reference for function implementations across all languages. Abstracts away all surrogate keys, junction tables, and JOIN complexity in order to return a clean, flat result that basically answers: "How is this function implemented in each language?"

Expected Results: One row per language implementation of a function, ordered alphabetically by function name, then by language name. Returns all descriptive columns needed for a complete reference entry.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`mcpmatt`@`localhost` SQL SECURITY
DEFINER
VIEW `vw_function_lookup` AS
  SELECT
    l.`language_name`,
    f.`function_name`,
    f.`category`,
    l.`version`,
    i.`syntax`,
    i.`example`,
    i.`result`,
    i.`notes`,
    i.`date_added`
  FROM `implementation` i
  JOIN `function_implementation` fi ON i.`implementation_id` =
fi.`implementation_id`
```

```
  JOIN `language`                 l  ON i.`language_id`      =
l.`language_id`
  JOIN `function_table`           f  ON fi.`function_id`     =
f.`function_id`
  ORDER BY f.`function_name` ASC, l.`language_name` ASC;
```

View 2: vw_operator_lookup

Purpose: Serves as the primary full-detail reference for operator implementations across all languages. Follows the same abstraction pattern as vw_function_lookup, but for the operator domain, and also exposes the symbol column which is specific to the operator junction table.

Expected Results: One row per language implementation of an operator, order alphabetically by operator name then language name. Returns all descriptive columns including the language-specific operator symbol.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`mcpmatt`@`localhost` SQL SECURITY
DEFINER
VIEW `vw_operator_lookup` AS
  SELECT
    l.`language_name`,
    o.`operator_name`,
    o.`category`,
    l.`version`,
    oi.`symbol`,
    i.`syntax`,
    i.`example`,
    i.`result`,
    i.`notes`,
    i.`date_added`
  FROM `implementation` i
  JOIN `operator_implementation` oi ON i.`implementation_id` =
oi.`implementation_id`
  JOIN `language`                l  ON i.`language_id`      =
l.`language_id`
  JOIN `operator`                o  ON oi.`operator_id`     =
o.`operator_id`
  ORDER BY o.`operator_name` ASC, l.`language_name` ASC;
```

View 3: vw_structure_lookup

Purpose: Serves as the primary full-detail reference for data structure implementations across all languages. It shares the same pattern of the other two full lookup views, covering the data structure domain.

Expected Results: One row per language implementation of a data structure, ordered alphabetically by structure name then language name. Returns all descriptive columns needed for a complete reference entry.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`mcpmatt`@`localhost` SQL SECURITY
DEFINER
VIEW `vw_structure_lookup` AS
  SELECT
    l.`language_name`,
    ds.`structure_name`,
    ds.`category`,
    l.`version`,
    i.`syntax`,
    i.`example`,
    i.`result`,
    i.`notes`,
    i.`date_added`
  FROM `implementation` i
  JOIN `structure_implementation` si ON i.`implementation_id` =
si.`implementation_id`
  JOIN `language`                 l  ON i.`language_id`       =
l.`language_id`
  JOIN `data_structure`           ds ON si.`structure_id`     =
ds.`structure_id`
  ORDER BY ds.`structure_name` ASC, l.`language_name` ASC;
```

View 4: vw_function_quick_reference

Purpose: A restrictive view derived from the function domain, designed for a quick-lookup use case where only the callable syntax is needed. Hides all overly detailed columns, making it easier for quick function lookups in a given language without additional explanation.

Expected Results: One row per language implementation of a function, ordered alphabetically by function name then language name. Returns a deliberately reduced result set of only four columns.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`mcpmatt`@`localhost` SQL SECURITY
DEFINER
VIEW `vw_function_quick_reference` AS
  SELECT
    l.`language_name`,
    f.`function_name`,
    l.`version`,
    i.`syntax`
  FROM `implementation` i
  JOIN `function_implementation` fi ON i.`implementation_id` =
fi.`implementation_id`
  JOIN `language`                l  ON i.`language_id`      =
l.`language_id`
  JOIN `function_table`          f  ON fi.`function_id`     =
f.`function_id`
  ORDER BY f.`function_name` ASC, l.`language_name` ASC;
```

View 5: vw_operator_category_summary

Purpose: A restrictive view derived from the operator domain, design for browsing and filtering operators by type without exposing any code detail. Ordered by category first to make it natural to scan all arithmetic, comparison, logical, and other operators grouped together. Appropriate for high-level overviews of what operators exist and how they are symbolized across languages, without implementation specifications.

```
CREATE ALGORITHM=UNDEFINED DEFINER=`mcpmatt`@`localhost` SQL SECURITY
DEFINER
VIEW `vw_operator_category_summary` AS
  SELECT
    l.`language_name`,
    o.`operator_name`,
    o.`category`,
    oi.`symbol`
  FROM `implementation` i
  JOIN `operator_implementation` oi ON i.`implementation_id` =
oi.`implementation_id`
```

```
  JOIN `language`                l  ON i.`language_id`       =
l.`language_id`
  JOIN `operator`                o  ON oi.`operator_id`      =
o.`operator_id`
  ORDER BY o.`category` ASC, o.`operator_name` ASC, l.`language_name` ASC;
```

2. Take screenshots showing the data displayed under each view.

View 1: vw_function_lookup

←T→	language_name	function_name	category	version	syntax	example	result	notes	date_added
Edit Copy Delete	JavaScript	filter	Functional	ES6+	array.filter(callback)	const scores = [85, 42, 93, 67, 55]; const passing...	[85, 93, 67]	Returns new array with elements that pass the test	2026-02-22
Edit Copy Delete	Python	filter	Functional	3.x	filter(function, iterable)	ages = [12, 18, 25, 8, 30] adults = list(filter(la...	[18, 25, 30]	Returns elements where function returns True	2026-02-22
Edit Copy Delete	C++	input	I/O	C++11	std::cin >> variable;	int age; std::cout << "Enter age: "; std::cin >> a...	User enters: 25 -> age = 25	Use getline(cin, str) for strings with spaces	2026-02-22
Edit Copy Delete	Java	input	I/O	Java 8+	Scanner sc = new Scanner(System.in); sc.nextLine()...	Scanner sc = new Scanner(System.in); String name =...	User enters: Alice -> name = "Alice"	Use nextInt(), nextDouble() for other types	2026-02-22
Edit Copy Delete	Python	input	I/O	3.x	variable = input(prompt)	age = int(input("Enter your age: "))	User enters: 25 -> age = 25	Always returns string; cast to int/float as needed	2026-02-22
Edit Copy Delete	JavaScript	join	String	ES6+	array.join(separator)	const tags = ["python", "javascript", "sql"]; tags...	python, javascript, sql	Default separator is comma	2026-02-22
Edit Copy Delete	Python	join	String	3.x	separator.join(iterable)	words = ["Hello", "World", "2024"] " ".join(words)	Hello World 2024	Separator is the string you call join on	2026-02-22
Edit Copy Delete	C++	len	Utility	C++11	container.size()	std::string name = "Alice"; name.size();	5	Returns size_t type; use .length() for strings too	2026-02-22
Edit Copy Delete	Java	len	Utility	Java 8+	array.length or string.length() or list.size()	int[] nums = {10, 20, 30, 40}; nums.length;	4	Arrays use .length (no parens), Strings use .lengt...	2026-02-22
Edit Copy Delete	Python	len	Utility	3.x	len(sequence)	words = ["apple", "banana", "cherry"] len(words)	3	Works on strings, lists, tuples, dicts, sets	2026-02-22
Edit Copy Delete	JavaScript	map	Functional	ES6+	array.map(callback)	const prices = [10, 20, 30]; const withTax = price...	[10.8, 21.6, 32.4]	Arrow functions make this concise	2026-02-22
Edit Copy Delete	Python	map	Functional	3.x	map(function, iterable)	nums = [1, 2, 3, 4] squared = list(map(lambda x: x...	[1, 4, 9, 16]	Returns iterator; wrap in list() to see results	2026-02-22
Edit Copy Delete	C++	print	I/O	C++11	std::cout << value << std::endl;	std::cout << "Temperature: " << 72 << "F" << std::...	Temperature: 72F	Chain multiple values with << operator	2026-02-22
Edit Copy Delete	Java	print	I/O	Java 8+	System.out.println(value);	System.out.println("Items in cart: " + 5);	Items in cart: 5	Use + for string concatenation	2026-02-22
Edit Copy Delete	JavaScript	print	I/O	ES6+	console.log(value);	console.log(`User ${name} logged in`);	User John logged in	Template literals use backticks and ${} for interp...	2026-02-22
Edit Copy Delete	Python	print	I/O	3.x	print(value, sep=" ", end="\n")	print("Score:", 95, sep=" ")	Score: 95	Supports multiple arguments with customizable sepa...	2026-02-22
Edit Copy Delete	C++	sort	Sorting	C++11	std::sort(begin, end)	std::vector<int> v = {5, 2, 8, 1}; std::sort(v.beg...	{1, 2, 5, 8}	Requires #include <algorithm>; average O(n log n)	2026-02-22
Edit Copy Delete	Java	sort	Sorting	Java 8+	Arrays.sort(array) or Collections.sort(list)	List<String> names = Arrays.asList("Zoe", "Amy", "...	[Amy, Max, Zoe]	Use Comparator for custom sorting	2026-02-22
Edit Copy Delete	Python	sort	Sorting	3.x	list.sort() or sorted(iterable)	prices = [29.99, 9.99, 49.99, 19.99] prices.sort()	[9.99, 19.99, 29.99, 49.99]	.sort() modifies in place; sorted() returns new li...	2026-02-22
Edit Copy Delete	JavaScript	split	String	ES6+	string.split(separator)	const path = "home/user/documents"; const parts = ...	["home", "user", "documents"]	Can use regex as separator	2026-02-22
Edit Copy Delete	Python	split	String	3.x	string.split(separator)	csv_line = "John,25,Engineer" fields = csv_line.sp...	["John", "25", "Engineer"]	Default separator is whitespace	2026-02-22

View 2: vw_operator_lookup

←T→	language_name	operator_name	category	version	symbol	syntax	example	result	notes	date_added
Edit Copy Delete	Java	Addition	Arithmetic	Java 8+	+	a + b	int hoursWorked = 8 + 6 + 9;	23	String + int auto-converts int to String	2026-02-22
Edit Copy Delete	Python	Addition	Arithmetic	3.x	+	a + b	total = 15.99 + 4.50 + 2.00	22.49	Also concatenates strings and lists	2026-02-22
Edit Copy Delete	Python	Assignment	Assignment	3.x	=	variable = value	name = "Alice" count = 0 count += 1 # augmented a...	name="Alice", count=1	Also: +=, -=, *=, /=, //=, **=, %=	2026-02-22
Edit Copy Delete	Java	Division	Arithmetic	Java 8+	/	a / b	int pages = 100 / 3; double exactPages = 100.0 / 3...	pages = 33, exactPages = 33.333...	int/int = int (truncated); use doubles for decimal...	2026-02-22
Edit Copy Delete	Python	Division	Arithmetic	3.x	/	a / b (float) or a // b (integer)	avg = 285 / 3 whole_avg = 285 // 3	avg = 95.0, whole_avg = 95	/ always returns float; // truncates toward negati...	2026-02-22
Edit Copy Delete	Python	Equal	Comparison	3.x	==	a == b	password = "secret123" is_correct = password == "s...	True	Compares value; use "is" to compare identity (same...	2026-02-22
Edit Copy Delete	Python	Greater Than	Comparison	3.x	>	a > b	age = 21 can_drink = age > 20	True	Works with numbers and strings (lexicographic)	2026-02-22
Edit Copy Delete	Java	Less Than	Comparison	Java 8+	<	a < b	int temperature = 32; boolean freezing = temperatu...	true	Returns boolean primitive	2026-02-22

Edit Copy Delete	C++	Logical AND	Logical	C++11	and / &&	a && b	bool inStock = true;
							bool hasCredit = true;
							bool c...
								true	Keyword "and" also works in C++	2026-02-22

Edit Copy Delete	Python	Logical AND	Logical	3.x	and / &&	a and b	age = 25 has_license = True can_drive = age >= 16 ...	True	Short-circuits: if first is False, second not eval...	2026-02-22
Edit Copy Delete	Java	Logical NOT	Logical	Java 8+	not / !	!a	boolean isEmpty = list.isEmpty(); boolean hasItems...	true if list has items	Prefix operator, highest precedence among logical ...	2026-02-22
Edit Copy Delete	Python	Logical NOT	Logical	3.x	not / !	not a	logged_in = False show_login_button = not logged_i...	True	Inverts boolean value	2026-02-22
Edit Copy Delete	JavaScript	Logical OR	Logical	ES6+	or / ||	a || b	const username = inputName || "Guest";	"Guest" if inputName is empty/null	Often used for default values; consider ?? for nul...	2026-02-22
Edit Copy Delete	Python	Logical OR	Logical	3.x	or / ||	a or b	is_weekend = day == "Saturday" or day == "Sunday"	True if day is Saturday	Short-circuits: if first is True, second not evalu...	2026-02-22
Edit Copy Delete	JavaScript	Modulus	Arithmetic	ES6+	%	a % b	const hour24 = 27 % 24;	3	Useful for wrapping values (clock, array index cyc...	2026-02-22
Edit Copy Delete	Python	Modulus	Arithmetic	3.x	%	a % b	is_even = 42 % 2 remainder = 17 % 5	is_even = 0, remainder = 2	Common for checking even/odd or cycling through in...	2026-02-22
Edit Copy Delete	C++	Multiplication	Arithmetic	C++11	*	a * b	double area = 3.14159 * radius * radius;	area of circle calculation	Watch for integer overflow with large numbers	2026-02-22
Edit Copy Delete	Python	Multiplication	Arithmetic	3.x	*	a * b	weekly_pay = 25.50 * 40	1020.0	Also repeats strings: "ha" * 3 = "hahaha"	2026-02-22
Edit Copy Delete	Python	Not Equal	Comparison	3.x	!=	a != b	status = "active" needs_attention = status != "act...	False	Returns True if values are different	2026-02-22
Edit Copy Delete	JavaScript	Strict Equal	Comparison	ES6+	===	a === b (strict) or a == b (loose)	"5" === 5 "5" == 5	false, true	Always prefer === to avoid type coercion bugs	2026-02-22
Edit Copy Delete	Python	Subtraction	Arithmetic	3.x	-	a - b	balance = 1000.00 balance = balance - 250.75	749.25	Also used for set difference: set1 - set2	2026-02-22

View 3: vw_structure_lookup

	language_name	structure_name	category	version	syntax	example	result	notes	date_added
Edit Copy Delete	Java	Dictionary	Associative	Java 8+	HashMap<K, V> map = new HashMap<>();	HashMap<String, Double> prices = new HashMap<>(); ...	{apple=1.5, banana=0.75}	O(1) average lookup; use TreeMap for sorted keys	2026-02-22
Edit Copy Delete	JavaScript	Dictionary	Associative	ES6+	const obj = {key: value}; or new Map()	const config = {theme: "dark", fontSize: 14}; conf...	{theme: "dark", fontSize: 14, language: "en"}	Use Map for non-string keys or frequent add/delete	2026-02-22
Edit Copy Delete	Python	Dictionary	Associative	3.x	my_dict = {key: value, ...}	student = {"name": "Emma", "gpa": 3.8, "year": 2} ...	{"name": "Emma", "gpa": 3.8, "year": 2, "major": "...	Keys must be immutable (str, int, tuple)	2026-02-22
Edit Copy Delete	C++	List	Linear	C++11	std::vector<type> name = {values};	std::vector<double> temps = {72.5, 68.0, 75.3}; te...	{72.5, 68.0, 75.3, 71.2}	Dynamic size, contiguous memory, #include <vector>	2026-02-22
Edit Copy Delete	Java	List	Linear	Java 8+	ArrayList<Type> list = new ArrayList<>();	ArrayList<String> colors = new ArrayList<>(); colo...	[red, blue]	Resizable array; use LinkedList for frequent inser...	2026-02-22
Edit Copy Delete	JavaScript	List	Linear	ES6+	const arr = [item1, item2];	const fruits = ["apple", "mango"]; fruits.push("ki...	["grape", "apple", "mango", "kiwi"]	push=end, unshift=start, pop=remove end, shift=rem...	2026-02-22
Edit Copy Delete	Python	List	Linear	3.x	my_list = [item1, item2, ...]	shopping = ["milk", "eggs", "bread"] shopping.appe...	["almond milk", "eggs", "bread", "butter"]	Mutable, ordered, allows duplicates, mixed types O...	2026-02-22
Edit Copy Delete	Java	Queue	Linear	Java 8+	Queue<Type> q = new LinkedList<>();	Queue<String> tickets = new LinkedList<>(); ticket...	next = "customer1", queue = [customer2]	offer/poll preferred over add/remove (no exception...	2026-02-22
Edit Copy Delete	Python	Queue	Linear	3.x	from collections import deque; q = deque()	from collections import deque printer_queue = dequ...	next_job = "doc1", queue = ["doc2", "doc3"]	Use deque for O(1) operations on both ends	2026-02-22
Edit Copy Delete	Java	Set	Collection	Java 8+	HashSet<Type> set = new HashSet<>();	HashSet<Integer> lottery = new HashSet<>(); lotter...	[7, 14]	No duplicates, no guaranteed order; use TreeSet fo...	2026-02-22
Edit Copy Delete	Python	Set	Collection	3.x	my_set = {item1, item2, ...}	tags = {"python", "coding", "python", "tutorial"} ...	{"python", "coding", "tutorial", "beginner"}	Duplicates automatically removed; unordered	2026-02-22
Edit Copy Delete	Java	Stack	Linear	Java 8+	Stack<Type> stack = new Stack<>();	Stack<String> history = new Stack<>(); history.pus...	back = "page2", stack = [page1]	Prefer Deque interface: ArrayDeque for better perf...	2026-02-22
Edit Copy Delete	Python	Stack	Linear	3.x	stack = [] # use append() and pop()	undo_stack = [] undo_stack.append("typed A") undo ...	last = "typed B", stack = ["typed A"]	Lists work as stacks; collections.deque is faster	2026-02-22

View 4: vw_function_quick_reference

	language_name	function_name	version	syntax
Edit Copy Delete	JavaScript	filter	ES6+	array.filter(callback)
Edit Copy Delete	Python	filter	3.x	filter(function, iterable)
Edit Copy Delete	C++	input	C++11	std::cin >> variable;
Edit Copy Delete	Java	input	Java 8+	Scanner sc = new Scanner(System.in); sc.nextLine()...
Edit Copy Delete	Python	input	3.x	variable = input(prompt)
Edit Copy Delete	JavaScript	join	ES6+	array.join(separator)
Edit Copy Delete	Python	join	3.x	separator.join(iterable)
Edit Copy Delete	C++	len	C++11	container.size()
Edit Copy Delete	Java	len	Java 8+	array.length or string.length() or list.size()
Edit Copy Delete	Python	len	3.x	len(sequence)
Edit Copy Delete	JavaScript	map	ES6+	array.map(callback)
Edit Copy Delete	Python	map	3.x	map(function, iterable)
Edit Copy Delete	C++	print	C++11	std::cout << value << std::endl;
Edit Copy Delete	Java	print	Java 8+	System.out.println(value);
Edit Copy Delete	JavaScript	print	ES6+	console.log(value);
Edit Copy Delete	Python	print	3.x	print(value, sep=" ", end="\n")
Edit Copy Delete	C++	sort	C++11	std::sort(begin, end)
Edit Copy Delete	Java	sort	Java 8+	Arrays.sort(array) or Collections.sort(list)
Edit Copy Delete	Python	sort	3.x	list.sort() or sorted(iterable)
Edit Copy Delete	JavaScript	split	ES6+	string.split(separator)
Edit Copy Delete	Python	split	3.x	string.split(separator)

View 5: vw_operator_category_summary

←T→	language_name	operator_name	category	symbol
Edit Copy Delete	Java	Addition	Arithmetic	+
Edit Copy Delete	Python	Addition	Arithmetic	+
Edit Copy Delete	Java	Division	Arithmetic	/
Edit Copy Delete	Python	Division	Arithmetic	/
Edit Copy Delete	JavaScript	Modulus	Arithmetic	%
Edit Copy Delete	Python	Modulus	Arithmetic	%
Edit Copy Delete	C++	Multiplication	Arithmetic	*
Edit Copy Delete	Python	Multiplication	Arithmetic	*
Edit Copy Delete	Python	Subtraction	Arithmetic	-
Edit Copy Delete	Python	Assignment	Assignment	=
Edit Copy Delete	Python	Equal	Comparison	==
Edit Copy Delete	Python	Greater Than	Comparison	>
Edit Copy Delete	Java	Less Than	Comparison	<
Edit Copy Delete	Python	Not Equal	Comparison	!=
Edit Copy Delete	JavaScript	Strict Equal	Comparison	===
Edit Copy Delete	C++	Logical AND	Logical	and / &&
Edit Copy Delete	Python	Logical AND	Logical	and / &&
Edit Copy Delete	Java	Logical NOT	Logical	not / !
Edit Copy Delete	Python	Logical NOT	Logical	not / !
Edit Copy Delete	JavaScript	Logical OR	Logical	or / ||
Edit Copy Delete	Python	Logical OR	Logical	or / ||

3. Include the SQL statements you would have executed to grant privileges on a view and create a role. Document each query's intended purpose, expected results, and actual SQL command.

SQL Statement 1: Grant Privileges on a View to a Specific User
Purpose: Creates a new MySQL user account with no privileges by default. "localhost" restricts the account to local connections only.
Expected Result: A new user account exists in the mysql.user system table with no granted permissions.
Command:
Create USER 'readonly_user'@'localhost'
IDENTIFIED BY 'SuperSecurePassword123!!';

SQL Statement 2: Grant SELECT on a specific view

Purpose: Gives user read-only access to the vw_function_quick_reference view specifically. It allows the user to run SELECT queries against it, but cannot access vw_function_lookup, the implementation table, or any other object in the database.
Expected Result: The user can successfully execute SELECT * FROM vw_function_quick_reference but will receive an Access Denied error on any other table or view.
Command:
GRANT SELECT ON adminName_databaseName.vw_function_quick_reference
TO ‘readonly_user’@’localhost’;

SQL Statement 3: Create a Role and Assign Users to it
Purpose: Defines a named role. A role has no password and cannot log in, it exists as a privilege container.
Expected Result: A new role entry appears in mysql.user with account_locked = ‘Y’, confirming it is not a login account.
Command:
CREATE ROLE ‘read_only_viewer’;

SQL Statement 4: Grant Privileges to the Role
Purpose: Assigns the specific view-level SELECT privileges to the role rather than to any individual user. Any user later assigned this role inherits both of these permissions automatically.
Expected Result: The role now carries SELECT rights on both restrictive views, visible through “SHOW GRANTS FOR ‘read_only_viewer’.
Command:
GRANT SELECT ON adminName_databaseName.vw_function_quick_reference
TO ‘read_only_viewer’;
GRANT SELECT ON adminName_databaseName.vw_operator_category_summary
TO ‘read_only_viewer’;

SQL Statement 5: Create Users to Assign the Role to
Purpose: Creates two separate user accounts that will receive access through the role rather than through direct grants.
Expected Results: Both accounts exist with no privileges of their own yet.
Command:
CREATE USER ‘student_user’@’localhost’ IDENTIFIED BY ‘StudentPass123!!’;
CREATE USER ‘instructor_user’@’localhost’ IDENTIFIED BY ‘InstructorPass123!!’;

SQL Statement 6: Assign the Role to Both Users

Purpose: Assigns the role to each user. THe users inherit all privileges the role contains, in this case: SELECT on both views.
Expected Result: Both users can now query vw_function_quick_reference and vw_operator_category_summary. If the role's privileges are ever updated, both users will reflect the change immediately with no additional actions needed.
Command:
GRANT 'read_only_viewer' TO 'student_user'@'localhost';
GRANT 'read_only_viewer' TO 'instructor_user'@'localhost';

4. Identify and explain each non-conforming element in your database schema. Describe how it violates normalization principles and why it needs to be addressed.

3NF requires that every non-key attribute depends on the key, the whole key, and nothing but the key. This means that there cannot be any non-key attributes that can be determined by another non-key attribute (a transitive dependency). In the implementation table, there is a non-conforming element, the version attribute. In the original schema, the implementation table carried a "version" column (e.g., '3.x', 'C++11', 'Java 8+', 'ES6+'). The functional dependency chain was:

implementation_id -> language_id -> version

This is an example of a transitive dependency. Here, version was not determined by implementation_id directly, it was actually determined by language_id, which is a non-key attribute of implementation. Basically, "version" told us something about the language, not about the specific implementation row. 3NF prohibits this pattern as version is being functionally determined by another non-key attribute rather than by the primary key and nothing but the primary key.

This must be addressed, as leaving a transitive dependency in place introduces three categories of anomaly:

- **Update Anomaly:** If Python's version designation changed from '3.x' to '3.12+', every single row in implementation belonging to Python would need to be updated individually. Missing even one row would leave the database in an inconsistent state.
- **Insertion Anomaly:** A new language could not have its version recorded until at least one implementation row existed for it, because "version" had no home of its own.

- **Deletion Anomaly:** If all implementation rows for a given language were deleted, the version information for that language would be permanently lost with them.

5. Provide a detailed explanation of changes made during the normalization process. Justify each modification, explaining how it improves database efficiency, eliminates redundancy and inconsistencies, and ensures 3NF compliance.

In order to ensure 3NF compliance, we began by applying the rules for 1NF and 2NF:

First Normal Form (1NF) - Fully Compliant

Every table in the schema satisfies all 1NF rules:

- **No row ordering dependency:** No table relies on physical row order to convey meaning. All data is identified by explicit primary keys.
- **No mixed data types:** Every column holds a single, consistent data type throughout (e.g., implementation_id is always INT, language_name is always VARCHAR).
- **Every table has a primary key:** All eight tables define an explicit PRIMARY KEY, either as a single integer or a composite key on the group tables.
- **No repeating groups or multi-valued attributes:** No column stores lists, comma-separated values, or arrays. Each cell holds exactly one atomic value.

Second Normal Form (2NF) - Full Compliant

2NF requires that every non-key attribute depends on the entire primary key, removing partial dependencies. For our database, this is only a concern in tables with composite primary keys, however, each table has no non-key attributes, so no partial dependency is possible. In every other table, each attribute depends on the entire primary key.

Third Normal Form (3NF) - One Violation Identified

As was explained in Question 4, 3NF requires that every non-key attribute depends on “the key, the whole key, and nothing but the key”. In order to address the previously identified dependency issues caused by the ‘value’ attribute, the following modification was made:

From:
implementation_id -> language_id -> version

To:
language_id -> version

‘Version’ is now a direct, non-transitive attribute of language_id, its true determinant. The ‘version’ column was removed from implementation, and all three views were updated to retrieve ‘version’ via the existing JOIN to language. This preserves the output with no impact to any queries written against the views.

Modification Efficiency Improvements:

Previously, every implementation row repeated the version string for its language. For example, every Python row stored ‘3.x’, every Java row stored ‘Java 8+’, and so on. WIth ‘version’ in the ‘language’ table, that string is stored exactly once per language and retrieved from the already necessary JOIN, adding no overhead while reducing storage as the dataset grows.

Redundancy and Inconsistency Elimination:

Under the original design, updating a language’s version required touching every implementation row belonging to that language. Missing even one row would leave the database in an inconsistent state where the same language reported different versions across rows. With ‘version’ in the ‘language’ table, a single UPDATE to one row updates universally and inconsistency is completely eliminated.